

Table for Four: A System of Five Agents

CMU AI Agent Certification · Capstone · Week 5 Author: Manish Bhatt

Table for Four is an MCP first restaurant concierge that carries a guest from one sentence, "somewhere Italian for four on Friday", to a confirmed reservation with a governance trail behind it. Its agents are divided by authority rather than by skill, a departure that section 2 argues for rather than assumes.

1. The problem, and why one agent is not enough

Booking a restaurant for someone else is not a retrieval problem. It is a sequence of small commitments made on a guest's behalf, only one of which is irreversible. The concierge must find candidates from a live source, match them against a perks catalogue, check real availability, learn and reuse what it knows about the guest, look up what the food is actually like, and then, once, write a booking a restaurant will hold a table for.

Those steps differ in **authority**, not merely in skill. Searching is free and reversible; writing to a guest's stored profile changes how they are treated on every future visit; creating a booking commits them to being somewhere at a time. One agent holding every capability has no structural way to express that difference. It can be *told* not to rewrite a standing preference while searching, and in this codebase that class of instruction has failed four separate times in ways a guest noticed. So the value of separating agents is not division of labour. It is that a capability a unit does not hold cannot be misused by a unit that reasons badly, and reasoning badly is the failure mode actually observed.

2. How many agents, and how that number was reached

Five. The number came from asking, of each capability, who should be able to do this and who must never, then merging any two that gave the same answer.

The obvious move is a planner, a researcher and a writer, each with its own model. Week 4 measured what that would cost: the conversational path has exactly **two model call sites**, bounded by `MAX_TOOL_HOPS = 6`. Every additional model bearing agent multiplies latency and tokens in a live conversation, buying judgement the problem does not need, because the hard constraints all have exact answers from tool calls. Whether 19:00 is free, whether the party fits, whether a booking is more than 24 hours away: these are function calls, not opinions.

The boundary is therefore drawn around capability, and exactly one unit carries a model. A sixth would need a sixth distinct answer to "who may do this?", and the diminishing return is immediate: a unit with no capability of its own is documentation with a filename.

3. Roles and responsibilities

Each unit is a markdown file whose frontmatter declares `tools` (what it may do), `never` (what it is denied), and `handlers` (the tools it answers).

Unit	Role	Holds	Cannot
Dino	Host, front of house, the only unit with a model	<i>nothing</i>	every capability in the system
Scout	Reservations desk: finds the table, never takes it	search, perks, availability	book, cancel, write to memory
Curator	Kitchen research: knows the food, never meets the guest	web highlights, photos	search, book, write to memory
Steward	Keeper of the book: the member's own record	remember, adopt an identity	search, book, reach the web
Booker	The pass: the only unit that commits the guest	create and cancel bookings	search, reach the web, adopt an identity

Two are the design in miniature. **Dino holds nothing at all:** the unit that runs the model has an empty grant and can only ask a unit that holds something. This inverts the usual arrangement, where the reasoning component owns every tool and is asked politely to behave; here misbehaviour is a `NotGranted` exception rather than a policy violation. **Booker cannot search**, so a booking may only name a restaurant Scout already surfaced, even when the shortlist is empty.

4. Coordination: a graph, with a human as a node

The orchestrator is an explicit **LangGraph state machine:** parse, search, refine, match perks, rank, propose, gate, book, audit. One loop returns refine to search, because a first attempt can be over constrained and needs a relaxed

retry, bounded at two iterations so it cannot spin. Two conditional edges make this a network rather than an assembly line: one routes a failed search to refine, the other routes the gate to either book or straight to the end.

The conversational surface is event driven instead. The guest speaks, Dino decides which unit to ask, and `_dispatch` runs that handler under its owning unit's grant. There is no fixed order, because a real conversation has none: a guest may name a restaurant outright and skip the shortlist entirely.

The decision I would most defend is that **the human is a node in the graph, not a caller of it**. `gate_node` issues a LangGraph interrupt: the run checkpoints and genuinely stops until a decision arrives from outside. Only an explicit approval resumes it; a malformed resume, a missing approver or an unattended run all decline. The irreversible step is the one no agent may take on its own say so.

5. Communication

Units never talk to each other. Every exchange is agent, then tool, then structured JSON, then agent, and the shape of that JSON carries much of the coordination.

Refusals that instruct. No handler fails silently. `book_table` alone can return eleven distinct statuses before it writes anything, each carrying the reason and the next action. This is one way communication behaving like validation, because the refusal is computed rather than judged.

Working memory, restated every turn. Every tool result carries a `known_so_far` block. Long conversations drift and a model will ask again for an email it was given twenty turns ago, so the facts travel with every result.

Withholding rather than instructing. When the guest has already named an available time, the availability result now contains only that time. Telling the model not to read the other options back did not work; removing them from the payload did. Where an instruction and a data shape disagree, the data shape wins.

The one genuinely two way channel is the approval gate, and it is two way with a person.

6. Trade offs

Reliability against latency. Deterministic checks were chosen over model judgement throughout. Every reply passes a grounding check: each time, date, confirmation id and email must trace to a tool result, or the sentence carrying it is removed before the guest sees it. A model judge would cover more, including dish names, which this deliberately does not. It would also cost a call, could be argued out of a refusal, and could not run offline.

Coordination overhead against expressiveness. The measured cost of five units with enforced grants is zero additional model calls and zero additional tokens: briefs are read from files already being sent, and a test pins the tool schema budget at 6,624 characters so the harness cannot become a second place to write prompt.

Bounded loops against thoroughness. Both loop guards cap work that could in principle continue: a demo which stalls is worse than one which offers a slightly weaker shortlist.

The trade off declined. Week 4 argues against search over generated branches, because the branches here are retrieved rather than reasoned and the guest is the search algorithm by design.

7. Scale, and the honest limit

Adding capability is local. One package per MCP server owns its fixtures and store, so deleting a server directory changes nothing else, and adding a unit means adding a file and a grant. When photo lookup was added this month the change was one line of frontmatter, a paragraph saying why, and a test asserting Booker still could not reach it.

Guarantees hold as the system grows because they sit at chokepoints rather than spread across call sites: `_dispatch` is the only route to a tool, `roster.require` the only route to memory, `gate_node` the only route to a booking. New code inherits them by construction, and the audit trail names the acting unit because that chokepoint already knew it.

The limit, plainly: this scales along the axis of authority far better than along the axis of reasoning. An evening across several venues, where travel time and closing hours interact, is combinatorial in a way this architecture does not address.

Grounding. Every claim is checkable in the repository, against `agent/roster/`, `agent/graph.py`, `governance/`, and 205 offline tests that need no API key.